

Calcolatori Elettronici

18 settembre 2024

Teoria

Esercizio 1 (11 punti): Progettare un circuito combinatorio che implementi una funzione logica con tre ingressi A , B , e C e un'uscita F . La funzione è definita come segue:

$$F(A, B, C) = (A \wedge \overline{B}) \vee (B \wedge C)$$

1. Disegnare il circuito logico utilizzando porte logiche elementari (AND, OR, NOT).
2. Scrivere la tabella di verità della funzione.
3. Semplificare la funzione utilizzando le mappe di Karnaugh, se possibile, e ridisegnare il circuito semplificato.

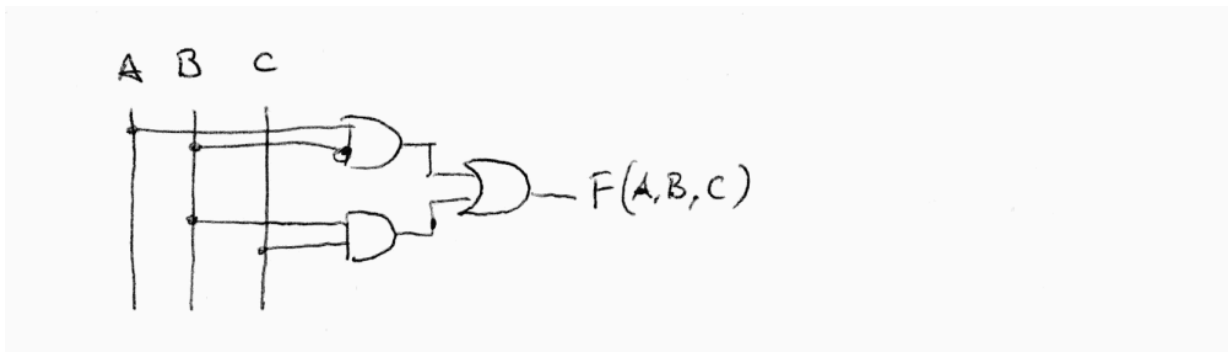
Esercizio 2 (8 punti): Calcolare la rappresentazione binaria, in complemento a due ad 8 bit, dei seguenti numeri decimali: -45 e 76 . Eseguire l'operazione di somma tra i due numeri nella loro rappresentazione binaria, mostrando tutti i passaggi. Specificare se si verifica un overflow. Convertire il risultato finale in decimale e verificare la correttezza dell'operazione.

Esercizio 3 (11 punti): Spiegare in dettaglio il funzionamento dell'architettura pipeline di un processore, descrivendo i principali stadi di esecuzione delle istruzioni. Come vengono gestiti i potenziali hazard nelle architetture moderne e quali tecniche vengono adottate per mitigare gli effetti negativi delle dipendenze dei dati, dei controlli e delle risorse? Illustra la risposta con esempi concreti.

Soluzioni

Esercizio 1

Il circuito equivalente dell'equazione proposta è facilmente ricavabile sostituendo gli operatori logici con delle porte logiche, rispettando le precedenze imposte dalle parentesi:



La tabella di verità può essere calcolata valutando la funzione con tutte le 8 combinazioni degli input:

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

Riguardo la minimizzazione: la funzione proposta è già in somma di prodotti, pertanto è già in forma canonica e quindi minimizzata. Ciò può essere comunque verificato costruendo la mappa di Karnaugh:

AB		C			
		00	01	11	10
C	0	0	0	0	1
	1	0	1	1	1

che fornisce come riduzione esattamente $F(A, B, C) = (A \wedge \overline{B}) \vee (B \wedge C)$.

Esercizio 2

Per determinare la rappresentazione di -45 , convertiamo prima il numero in base 2, con divisioni successive. Si ottiene $45_{10} = 00101101_2$. Per calcolare il complemento a due, calcoliamo prima il complemento a uno invertendo tutti i bit, ottenendo: $00101101_2 \rightarrow 11010010_2$. A questo punto sommiamo 1: $11010010_2 + 1 = 11010011_2$, ottenendo che $-45_{10} = 11010011_2$.

Per 76 possiamo direttamente convertire il valore in binario con divisioni successive, ottenendo $76_{10} = 01001100_2$.

La somma si ottiene effettuando la somma bit a bit:

1	1	0	1	0	0	1	1
0	1	0	0	1	1	0	0
0	0	0	1	1	1	1	1

con ultimo riporto pari a 1 e penultimo riporto pari a 1. Poiché gli ultimi due riporti sono concordi, non vi è overflow. Ciò era prevedibile poiché, essendo gli operandi di segno opposto, il risultato è sempre rappresentabile. Infatti: $00011111_2 = 31_{10} = -45 + 76$.

Progetto

Un processore `z64` controlla un firewall, composto dalle periferiche `RICEZIONE` e `TRASMISSIONE`. La periferica ricezione invia una richiesta di interruzione al processore ogni volta che è disponibile un nuovo pacchetto dati in ricezione. Mediante un apposito registro di interfaccia, il processore è in grado di conoscere l'indirizzo IP del mittente del pacchetto (una longword).

In memoria, è presente il vettore `denylist` che contiene una sequenza di IP di mittenti (ciascuno sempre memorizzato come longword) dei quali si vuole scartare qualsiasi pacchetto in ingresso. Lo `z64` verifica se l'IP del pacchetto attualmente in ricezione proviene da uno di questi indirizzi. In caso affermativo, programma nuovamente `RICEZIONE` per ricevere un nuovo pacchetto—`RICEZIONE` non riceve pacchetti se non viene esplicitamente avviato dallo `z64`.

In caso negativo, lo `z64` acquisisce un pacchetto di dati da `RICEZIONE`. Un pacchetto ha una dimensione pari a 65 kB. la periferica `RICEZIONE` è anche connessa al DMAC di sistema: lo `z64`, quindi, programma il DMAC per trasferire in memoria, in un apposito buffer, il pacchetto in ricezione.

Una volta completato il trasferimento, lo `z64` inoltra i dati ricevuti alla periferica `TRASMISSIONE`. Questa opera in modalità sincrona: i 65 kB vengono trasmessi utilizzando un apposito registro di interfaccia, una longword alla volta.

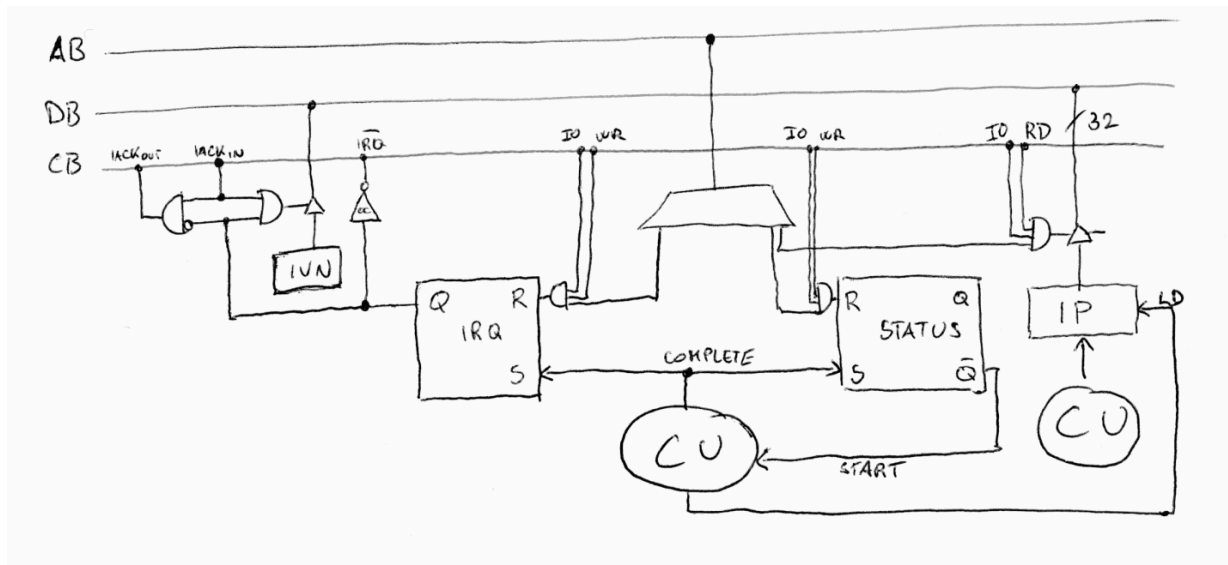
Attenzione: è importante differire la ricezione di un nuovo pacchetto fino al momento in cui il precedente è stato inoltrato completamente, per evitare sovrascritture.

Realizzare:

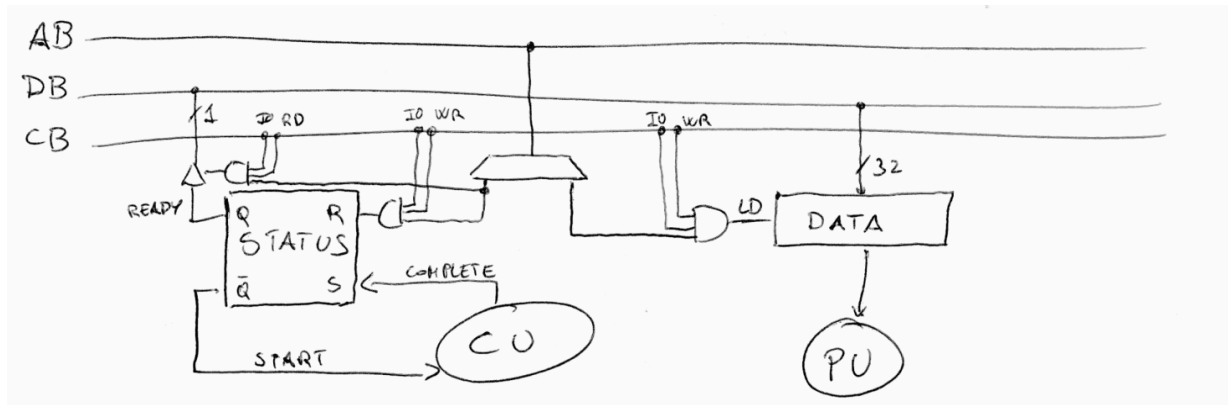
- Le interfacce dei dispositivi `RICEZIONE` (solo la parte di interfaccia con lo `z64`), e `TRASMISSIONE`;
- Tutto codice necessario al funzionamento del sistema, senza l'utilizzo di pseudo-istruzioni.

Soluzione

La periferica `RICEZIONE` (rispetto all'interfacciamento con lo `z64`) deve fornire un registro di input a 32 bit per poter recuperare l'indirizzo IP corrente. Lavorando con le richieste di interruzione, è necessario un flip flop di IRQ. Poiché il dispositivo non deve generare interruzioni fino a che il trasferimento verso `TRASMISSIONE` non è completato, viene inserito un flip flop di `STATUS`. La presenza di questo flip flop, al posto dell'uso di un solo flip flop per sincronizzare il funzionamento dei due dispositivi, permette di uscire dal driver senza dover implicitamente riavviare il dispositivo. In questo modo è possibile suddividere correttamente il codice in una top half e una bottom half.



TRASMISSIONE richiede un interfacciamento standard per lavorare in modo sincrono, con un registro di input a 32 bit:



Una possibile implementazione del software è la seguente.

```

1  .org 0x800
2  .data
3      .equ RECEIVE_IRQ, 0x0
4      .equ RECEIVE_STATUS, 0x1
5      .equ RECEIVE_MEM, 0x2      # Per consentire al DMAC il trasferimento
6      .equ RECEIVE_IP, 0x3
7      .equ TRANSMIT_STATUS, 0x4
8      .equ TRANSMIT_DATA, 0x5
9      denylist_size: .quad 3      # Numero di IP nella denylist
10     denylist: .fill 3, 4, 0      # Si assume preinizializzato
11     buffer: .fill 65536/8, 8, 0
12     packet_to_transfer: .byte 0 # Flag di sincronizzazione
13     ip_addr: .long 0            # IP recuperato da RICEZIONE
14 .text
15 main:
16     sti
17     .loop:                        # Verifica se è arrivato un pacchetto
18         cmpb $1, packet_to_transfer
19         jnz .loop
20         movl ip_addr, %edi        # Controlla se l'IP è in deny list (bottom half)
21         call check_denylist
22         test %rax, %rax

```

```

23     jnz .skip_packet
24     call do_transfer
25 .skip_packet:
26     movb $0, packet_to_transfer
27     outb %al, $RECEIVE_STATUS # Permetti la ricezione di un nuovo pacchetto
28     jmp .loop
29
30 check_denylist:
31     xorq %rcx, %rcx
32 .compare_loop:
33     cmpl denylist(,%rcx,4), %edi
34     jz .deny_found
35     addq $1, %rcx
36     cmpq denylist_size, %rcx
37     jnz .compare_loop
38     xorq %rax, %rax # Non trovato: ritorna false
39     ret
40 .deny_found:
41     movq $1, %rax # Trovato: ritorna true
42     ret
43
44 do_transfer:
45     movq $buffer, %rdi # Dove deve scrivere il DMAC
46     movq %rdi, %rsi # Stesso indirizzo, per il successivo trasferimento
47     movq $8192, %rcx # Numero di quadword da trasmettere (65KB / 8 = 16384)
48     movw $RECEIVE_MEM, %dx # L'I/O port da cui leggere i dati
49     cld
50     insq # Carica i dati tramite DMAC
51     movq $16384, %rcx # Numero di longword da trasmettere (65KB / 4 = 16384)
52 .transmit_loop:
53     movl (%rsi), %eax # Carica la longword corrente in %rax
54     outl %eax, $TRANSMIT_DATA # Trasmette la longword a TRASMISSIONE
55     outb %al, $TRANSMIT_STATUS
56 .bw:
57     inb $TRANSMIT_STATUS, %al
58     btb $0, %al
59     jnc .bw
60     addq $4, %rsi # Avanza al prossimo blocco di 4 byte
61     subq $1, %rcx
62     jnz .transmit_loop
63     ret
64
65 .driver 0 # RICEZIONE
66     outb %al, $RECEIVE_IRQ
67     movb $1, packet_to_transfer # top half
68     inl $RECEIVE_IP, %eax # bottom half
69     movl %eax, ip_addr
70     iret

```

Prova di laboratorio

Implementare in C una lista singolarmente concatenata che rappresenta una collezione di numeri interi. Il programma dovrà permettere all'utente di:

1. Inserire un nuovo numero alla fine della lista.
2. Cancellare un numero specificato dall'utente.
3. Visualizzare l'intero contenuto della lista.

Scrivere il codice completo del programma, inclusa la gestione della memoria e l'interazione con l'utente per inserire, cancellare e stampare i numeri della lista. Assicurarsi di gestire correttamente i casi limite, come l'eliminazione di un valore da una lista vuota o l'inserimento del primo nodo in una lista non inizializzata.

Soluzione

La seguente è una possibile soluzione all'esercizio.

```
1  #include <errno.h>
2  #include <limits.h>
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  struct nodo {
7      int valore;
8      struct nodo *prossimo;
9  };
10
11 void inserisci_fine(struct nodo **testa, int valore)
12 {
13     struct nodo *nuovo_nodo = malloc(sizeof(struct nodo));
14     if(nuovo_nodo == NULL) {
15         printf("Errore nell'allocazione della memoria\n");
16         return;
17     }
18     nuovo_nodo->valore = valore;
19     nuovo_nodo->prossimo = NULL;
20
21     while(*testa != NULL) {
22         testa = &((*testa)->prossimo);
23     }
24     *testa = nuovo_nodo;
25 }
26
27 void cancella_valore(struct nodo **testa, int valore)
28 {
29     while(*testa != NULL) {
30         if((*testa)->valore == valore) {
31             struct nodo *temp = *testa;
32             *testa = (*testa)->prossimo;
33             free(temp);
34             printf("Valore %d cancellato\n", valore);
35             return;
36         }
37         testa = &((*testa)->prossimo);
38     }
39     printf("Valore %d non trovato nella lista\n", valore);
40 }
41
42 void stampa_lista(struct nodo *testa)
43 {
44     printf("Contenuto della lista: ");
45     while(testa != NULL) {
46         printf("%d ", testa->valore);
47         testa = testa->prossimo;
48     }
49     printf("\n");
50 }
51
52 void libera_lista(struct nodo **testa)
53 {
54     while(*testa != NULL) {
55         struct nodo *temp = *testa;
56         *testa = (*testa)->prossimo;
57         free(temp);
58     }
59 }
60
61 int leggi_intero(const char* prompt) {
```

```

62     char buffer[256];
63     char *endptr;
64     long valore;
65
66     while (1) {
67         printf("%s", prompt);
68         if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
69             printf("Errore nella lettura dell'input.\n");
70             continue;
71         }
72         errno = 0;
73         valore = strtol(buffer, &endptr, 10);
74
75         if (errno != 0 || endptr == buffer || *endptr != '\n' || valore < INT_MIN || valore >
INT_MAX) {
76             continue;
77         }
78         return (int)valore;
79     }
80 }
81
82 int main() {
83     struct nodo* lista = NULL;
84     int scelta, valore;
85
86     do {
87         printf("1. Inserisci un numero\n");
88         printf("2. Cancella un numero\n");
89         printf("3. Visualizza lista\n");
90         printf("4. Esci\n");
91         scelta = leggi_intero("Scegli un'opzione: ");
92
93         switch (scelta) {
94             case 1:
95                 valore = leggi_intero("Inserisci il numero: ");
96                 inserisci_fine(&lista, valore);
97                 break;
98             case 2:
99                 valore = leggi_intero("Inserisci il numero da cancellare: ");
100                 cancella_valore(&lista, valore);
101                 break;
102             case 3:
103                 stampa_lista(lista);
104                 break;
105             case 4:
106                 libera_lista(&lista);
107                 printf("Uscita dal programma.\n");
108                 break;
109             default:
110                 printf("Opzione non valida!\n");
111         }
112     } while (scelta != 4);
113
114     return 0;
115 }

```

